

The Linux Completely Fair Scheduler

Theory, mathematics, and implementation

August 2026

Contents

1	A note on versions	2
2	The problem CFS solves	3
2.1	What came before	3
2.2	The model	3
3	The mathematics of proportional fairness	3
3.1	Generalized Processor Sharing	3
3.2	Virtual time	4
3.3	vruntime in the kernel	4
3.4	Why smallest-vruntime-first yields proportional fairness	5
3.5	The fairness bound	5
4	Weights and nice levels	6
4.1	The weight table	6
4.2	Why 1.25	6
4.3	Reciprocal multipliers	6
5	Data structures	7
5.1	The scheduling entity	7
5.2	The runqueue	8
5.3	The red-black tree	8
5.4	Wraparound: why the signed cast matters	9
5.5	min_vruntime	9
6	Timeslices, periods, and preemption	10
6.1	The scheduling period	10
6.2	Per-entity slice	11
6.3	Accounting: update_curr()	12
6.4	Tick-driven preemption	12
6.5	Wakeup preemption	13
6.6	Buddies	14
7	Sleepers, forks, and migration	15
7.1	The problem	15
7.2	Fork: START_DEBIT	15
7.3	Wake: bounded sleeper credit	16
7.4	Migration normalization	16

8	PELT: Per-Entity Load Tracking	16
8.1	The geometric series	17
8.2	Fixed-point implementation	17
8.3	The three signals	17
8.4	Entry points	18
9	Group scheduling	18
9.1	Motivation	18
9.2	The recursion	18
9.3	The group weight problem, and its math	19
10	Bandwidth control	20
10.1	Model	20
10.2	Mechanism	20
10.3	The practical trap	20
11	SMP: load balancing and task placement	21
11.1	Scheduling domains	21
11.2	Wakeup placement	21
11.3	Periodic balancing	21
11.4	NUMA balancing	22
12	Observability	22
13	EEVDF: what replaced CFS in 6.6	22
13.1	Why CFS was replaced	22
13.2	Lag and eligibility	23
13.3	Virtual deadlines	24
13.4	The augmented tree	24
13.5	Lag preservation	24
13.6	What changed for users	24
14	Complexity summary	25
15	Reading path	25
15.1	Papers	25
15.2	LWN articles worth finding	26

1 A note on versions

CFS was merged in Linux 2.6.23 (October 2007) and was the scheduler for `SCHED_NORMAL` / `SCHED_BATCH` tasks for sixteen years. **As of Linux 6.6 (October 2023), the core picking algorithm was replaced by EEVDF** (Earliest Eligible Virtual Deadline First), authored by Peter Zijlstra.

The name “fair class” and the file `kernel/sched/fair.c` survive, and most of the surrounding machinery — weights, virtual time, PELT load tracking, group scheduling, bandwidth control, load balancing — is unchanged. What changed is *which* runnable entity gets picked and *how* latency is expressed.

This document covers classic CFS in depth (Sections 2–12), because that is what was asked for and because it is the conceptual foundation, then covers what EEVDF changed (Section 13). Where source is cited, it is cited by file and function name rather than line number, since line numbers rot immediately.

Primary files:

Path	Contents
kernel/sched/fair.c	The scheduler class itself: ~13k lines
kernel/sched/sched.h	struct cfs_rq, struct rq, struct task_group, helpers
include/linux/sched.h	struct sched_entity, struct sched_avg, struct task_struct
kernel/sched/core.c	__schedule(), pick_next_task(), the nice-to-weight tables
kernel/sched/pelt.c, pelt.h	Per-Entity Load Tracking
kernel/sched/features.h	Toggleable scheduler heuristics
kernel/sched/debug.c	/proc/sched_debug, /sys/kernel/debug/sched/
Documentation/scheduler/sched-design-CFS.rst	Molnar's original design note

2 The problem CFS solves

2.1 What came before

The $O(1)$ scheduler (2.6.0–2.6.22) maintained 140 priority-indexed runqueues with active and expired arrays, and used a bitmap to find the highest non-empty priority in constant time. Timeslices were computed from static priority, and interactivity was inferred by a heuristic that tracked sleep time and awarded a dynamic priority bonus of up to five nice levels in either direction.

That heuristic was the problem. It was a pile of tuned constants (MAX_SLEEP_AVG, CHILD_PENALTY, PARENT_PENALTY, INTERACTIVE_DELTA) that classified tasks as “interactive” or “CPU-bound,” and it could be gamed, it mispredicted, and every fix for one workload regressed another.

Con Kolivas's RSDL (Rotating Staircase Deadline) scheduler demonstrated that you could delete the interactivity estimator entirely and get better desktop behaviour from a pure fairness policy. CFS took that lesson and rebuilt around an explicit fairness model.

2.2 The model

Ingo Molnar's framing: CFS models an “**ideal, precise multi-tasking CPU**” — a hypothetical machine that runs all n runnable tasks *simultaneously*, each at $1/n$ of full speed. On such a machine fairness is exact and instantaneous; there is no scheduling latency because nothing ever waits.

Real hardware runs one task per core at a time. So CFS does the next best thing: it tracks, for every task, how far behind the ideal machine that task has fallen, and always runs the one that is furthest behind.

That single sentence is the whole algorithm. Everything else is bookkeeping, weighting, and the machinery to make “furthest behind” cheap to compute.

3 The mathematics of proportional fairness

3.1 Generalized Processor Sharing

The formal object CFS approximates is **GPS** (Generalized Processor Sharing), from the network-queueing literature (Parekh and Gallager, 1993). Each task i has a positive weight w_i . Let $A(t)$ be the set of runnable tasks at time t . GPS defines the instantaneous service rate

$$r_i(t) = \frac{w_i}{\sum_{j \in A(t)} w_j}, \quad i \in A(t),$$

and $r_i(t) = 0$ for tasks that are not runnable. The cumulative service delivered to task i is the integral

$$S_i(t) = \int_0^t r_i(\tau) d\tau.$$

GPS is *fluid*: it is defined for infinitesimal quanta, so it is unimplementable on a real CPU. Every practical scheduler is a **packetized** approximation, and its quality is measured by how far its service curve deviates from the GPS curve.

For any two tasks continuously runnable over $[t_0, t_1]$, GPS satisfies exactly

$$\frac{S_i(t_1) - S_i(t_0)}{S_j(t_1) - S_j(t_0)} = \frac{w_i}{w_j}.$$

This is the property CFS wants.

3.2 Virtual time

The trick that makes GPS tractable is a change of variable. Define a **virtual time** function whose rate is normalized by the system's total runnable weight:

$$\frac{dV}{dt} = \frac{1}{\sum_{j \in A(t)} w_j}.$$

Under GPS each active task accumulates service at exactly $w_i dV$. So if we track, per task, the quantity

$$v_i = \frac{S_i}{w_i}$$

— its *virtual service* — then **all active tasks under GPS have identical virtual service at all times**, and that common value is $V(t)$. Fairness becomes an *equality invariant* rather than a ratio to be maintained.

That is the insight. The kernel's `vruntime` is exactly v_i , scaled by a constant.

3.3 vruntime in the kernel

CFS defines, for a task executing over a physical interval δ ,

$$\Delta \text{vruntime}_i = \delta \cdot \frac{\text{NICE_0_LOAD}}{\text{se->load.weight}} = \delta \cdot \frac{1024}{w_i}.$$

`NICE_0_LOAD` is 1024 (`include/linux/sched/prio.h`), so a nice-0 task's `vruntime` advances at exactly wall-clock rate while it runs. A high-priority (large weight) task's `vruntime` advances *slowly*; a low-priority task's advances *quickly*.

The policy is then trivially stated:

Always run the runnable entity with the smallest `vruntime`.

3.4 Why smallest-`vruntime`-first yields proportional fairness

Suppose the scheduler could switch infinitely often. Then it holds all runnable `vruntime` values equal to a common value v , because any task that drifted below would immediately be selected and pushed back up. From the definition,

$$\text{vruntime}_i = v \implies S_i = \frac{v w_i}{1024},$$

and therefore for any pair i, j ,

$$\frac{S_i}{S_j} = \frac{w_i}{w_j}. \quad \blacksquare$$

Exact GPS. The infinite-switching limit is unreachable, so the real question is the error bound.

3.5 The fairness bound

Let Δv be the maximum spread in `vruntime` across runnable tasks. CFS bounds this spread by preempting the running task once it has exceeded its slice (Section 6). If the largest virtual slice granted to any entity is g , then for two continuously-runnable tasks

$$|v_i - v_j| \leq g.$$

Substituting $v_i = 1024 S_i / w_i$ gives

$$\left| \frac{S_i}{w_i} - \frac{S_j}{w_j} \right| \leq \frac{g}{1024}.$$

This is the classic GPS **relative fairness bound**: normalized service between any two tasks never diverges by more than one granularity's worth of virtual time. The *lag* of a task — the standard measure of scheduler service error —

$$\text{lag}_i(t) = S_i^{\text{GPS}}(t) - S_i^{\text{CFS}}(t)$$

is likewise bounded by $O(g w_i / 1024)$. Making g small tightens fairness and shortens latency, at the cost of more context switches. `sysctl_sched_min_granularity` (default 0.75 ms, scaled) is exactly the knob that sets this tradeoff, and it exists because context switches are not free: TLB and cache pollution mean the useful-work-per-switch curve turns over well before $g \rightarrow 0$.

Note that classic CFS does **not** bound lag tightly on the negative side for sleeping tasks — a task that sleeps a long time accumulates arbitrary GPS credit that CFS refuses to honour. Section 7 covers how CFS clamps this, and Section 13 covers how EEVDF fixed it properly.

4 Weights and nice levels

4.1 The weight table

Nice values run from -20 (highest priority) to $+19$. CFS maps them to weights via a geometric series with ratio 1.25:

$$w(\text{nice}) \approx \frac{1024}{1.25^{\text{nice}}}.$$

The table lives in `kernel/sched/core.c` as `sched_prio_to_weight[40]`:

```
const int sched_prio_to_weight[40] = {
/* -20 */    88761,    71755,    56483,    46273,    36291,
/* -15 */    29154,    23254,    18705,    14949,    11916,
/* -10 */    9548,    7620,    6100,    4904,    3906,
/* -5 */     3121,    2501,    1991,    1586,    1277,
/* 0 */      1024,     820,     655,     526,     423,
/* 5 */       335,     272,     215,     172,     137,
/* 10 */      110,      87,      70,      56,      45,
/* 15 */       36,      29,      23,      18,      15,
};
```

4.2 Why 1.25

The design goal is that changing nice by one level changes a task's CPU share by roughly 10%, *relatively and cumulatively from any starting point* — the effect should be the same whether you go from nice 0 to 1 or from nice 10 to 11. That property requires a geometric table, not a linear one (the O(1) scheduler's linear timeslice mapping badly failed it).

Verify with two competing tasks at nice 0 and nice 1:

$$\text{share}(0) = \frac{1024}{1024 + 820} = 55.5\%, \quad \text{share}(1) = \frac{820}{1024 + 820} = 44.5\%.$$

One task gains about 10 percentage points, the other loses about 10, and the ratio between them is 1.25 — roughly a 25% relative gap, which is the documented intent.

The dynamic range is enormous by design: nice -20 against nice $+19$ is 88761 : 15, a ratio of about 5900 : 1.

4.3 Reciprocal multipliers

Dividing by weight on every accounting update would be expensive — the kernel avoids 64-bit division in hot paths. So there is a companion table `sched_prio_to_wmult[40]` holding $2^{32}/w$:

```
const u32 sched_prio_to_wmult[40] = {
/* -20 */    48388,    59856,    76040,    92818,    118348,
/* -15 */    147320,    184698,    229616,    287308,    360437,
...
/* 0 */      4194304,    5237765,    6557202,    8165337,    10153587,
...
};
```

`set_load_weight()` in `core.c` populates `p->se.load.weight` and `p->se.load.inv_weight` from these when a task's priority is set.

The core arithmetic helper is `__calc_delta()` in `fair.c`, which computes $\delta_{\text{exec}} \cdot w / \text{lw.weight}$ as a multiply-and-shift:

```
static u64 __calc_delta(u64 delta_exec, unsigned long weight,
                      struct load_weight *lw)
{
    u64 fact = scale_load_down(weight);
    u32 fact_hi = (u32)(fact >> 32);
    int shift = WMULT_SHIFT;          /* 32 */
    int fs;

    __update_inv_weight(lw);
    /* ... overflow-avoiding renormalization of fact and shift ... */
    fact = mul_u32_u32(fact, lw->inv_weight);
    /* ... */
    return mul_u64_u32_shr(delta_exec, fact, shift);
}
```

and the wrapper implementing the equation of Section 3.3:

```
static inline u64 calc_delta_fair(u64 delta, struct sched_entity *se)
{
    if (unlikely(se->load.weight != NICE_0_LOAD))
        delta = __calc_delta(delta, NICE_0_LOAD, &se->load);
    return delta;
}
```

Note the fast path: a nice-0 task skips the arithmetic entirely, since $\delta \cdot 1024/1024 = \delta$. That is the common case.

5 Data structures

5.1 The scheduling entity

CFS does not schedule tasks directly; it schedules `struct sched_entity`, which may represent a task or a group of tasks (Section 9). This is what makes hierarchical group scheduling fall out almost for free. From `include/linux/sched.h`:

```
struct sched_entity {
    struct load_weight      load;
    struct rb_node          run_node;
    u64                    deadline;      /* EEVDF */
    u64                    min_vruntime;  /* EEVDF: subtree min */
    struct list_head        group_node;
    unsigned int            on_rq;

    u64                    exec_start;
    u64                    sum_exec_runtime;
    u64                    vruntime;
    u64                    prev_sum_exec_runtime;

    u64                    nr_migrations;

#ifdef CONFIG_FAIR_GROUP_SCHED
    int                    depth;
    struct sched_entity    *parent;
    struct cfs_rq          *cfs_rq;      /* rq this is queued on */
#endif
}
```

```

    struct cfs_rq          *my_q;          /* rq owned by this group */
#endif
    struct sched_avg       avg;            /* PELT */
};

```

`vruntime` is the ordering key. `sum_exec_runtime` is honest nanoseconds of CPU consumed (what `/proc/PID/schedstat` and CPU accounting report). `prev_sum_exec_runtime` is snapshotted at every context-switch-in so that `check_preempt_tick()` can measure how long the *current* stint has lasted.

5.2 The runqueue

From `kernel/sched/sched.h`:

```

struct cfs_rq {
    struct load_weight    load;            /* sum of runnable weights */
    unsigned int          nr_running;
    unsigned int          h_nr_running;    /* including nested groups */

    u64                   exec_clock;
    u64                   min_vruntime;

#ifdef CONFIG_SCHED_CORE
    unsigned int          forceidle_seq;
    u64                   min_vruntime_fi;
#endif
    struct rb_root_cached tasks_timeline;

    struct sched_entity   *curr;
    struct sched_entity   *next;
    struct sched_entity   *last;
    struct sched_entity   *skip;

    struct sched_avg      avg;            /* PELT aggregates */
    /* ... group scheduling and bandwidth fields ... */
};

```

5.3 The red-black tree

Runnable entities live in `tasks_timeline`, a `struct rb_root_cached` — a red-black tree that additionally caches a pointer to its leftmost node, keyed on `vruntime`:

- **Pick next:** $O(1)$. It is the cached leftmost node.
- **Enqueue / dequeue:** $O(\log n)$.
- The currently-running entity is **not** in the tree. It is dequeued on `set_next_entity()` and reinserted on `put_prev_entity()`, which is why `cfs_rq->curr` is tracked separately.

```

struct sched_entity *__pick_first_entity(struct cfs_rq *cfs_rq)
{
    struct rb_node *left = rb_first_cached(&cfs_rq->tasks_timeline);
    if (!left)
        return NULL;
    return rb_entry(left, struct sched_entity, run_node);
}

```

Insertion uses `entity_before()` as the comparator, and `rb_add_cached()` maintains the leftmost pointer:

```
static inline bool entity_before(const struct sched_entity *a,
                                const struct sched_entity *b)
{
    return (s64)(a->vruntime - b->vruntime) < 0;
}
```

5.4 Wraparound: why the signed cast matters

vruntime is a u64 of virtual nanoseconds. It wraps after roughly 584 years of wall clock at nice 0 — but a nice +19 task’s vruntime advances 68 times faster, and more importantly min_vruntime is initialized in some paths to (u64)(- (1LL << 20)), deliberately near the top of the range, precisely so that wraparound is exercised early and often in testing rather than lurking as a 500-year bug.

So *every* vruntime comparison in fair.c is done as a signed difference, never as a direct <:

```
static inline u64 max_vruntime(u64 max_vruntime, u64 vruntime)
{
    s64 delta = (s64)(vruntime - max_vruntime);
    if (delta > 0)
        max_vruntime = vruntime;
    return max_vruntime;
}
```

The expression (s64)(a - b) < 0 is correct across the wrap boundary as long as the true difference is less than 2^{63} ns (about 292 years), which it always is. A naive $a < b$ would invert the ordering the moment one task’s vruntime wrapped and another’s had not. This idiom is used consistently and is worth internalizing before reading the code.

5.5 min_vruntime

cfs_rq->min_vruntime is a **monotonically non-decreasing floor** on the runqueue’s virtual time. It serves two purposes: it is the reference point for placing new and waking tasks (Section 7), and it is the basis for normalizing vruntime across CPU migration.

```
static void update_min_vruntime(struct cfs_rq *cfs_rq)
{
    struct sched_entity *curr = cfs_rq->curr;
    struct rb_node *leftmost = rb_first_cached(&cfs_rq->tasks_timeline);
    u64 vruntime = cfs_rq->min_vruntime;

    if (curr) {
        if (curr->on_rq)
            vruntime = curr->vruntime;
        else
            curr = NULL;
    }

    if (leftmost) {
        struct sched_entity *se = __node_2_se(leftmost);
        if (!curr)
            vruntime = se->vruntime;
        else
            vruntime = min_vruntime(vruntime, se->vruntime);
    }

    /* ensure we never gain time by being placed backwards. */
}
```

```
cfs_rq->min_vruntime = max_vruntime(cfs_rq->min_vruntime, vruntime);
}
```

The final `max_vruntime()` enforces monotonicity. Without it, an empty runqueue receiving a task with a small `vruntime` could rewind virtual time and hand that task unbounded credit.

6 Timeslices, periods, and preemption

CFS has no fixed timeslice. It computes one on demand from the target latency and the weight distribution.

6.1 The scheduling period

```
static u64 __sched_period(unsigned long nr_running)
{
    if (unlikely(nr_running > sched_nr_latency))
        return nr_running * sysctl_sched_min_granularity;
    else
        return sysctl_sched_latency;
}
```

The parameters, with defaults before CPU-count scaling:

Tunable	Default	Meaning
<code>sysctl_sched_latency</code>	6 ms	Target period in which every runnable task runs at least once
<code>sysctl_sched_min_granularity</code>	0.75 ms	Floor on any task's slice
<code>sched_nr_latency</code>	8	latency divided by <code>min_granularity</code>
<code>sysctl_sched_wakeup_granularity</code>	1 ms	Preemption hysteresis on wakeup

Written out, with P the period and n the runnable count:

$$P(n) = \begin{cases} \text{sched_latency}, & n \leq \text{sched_nr_latency} \\ n \cdot \text{min_granularity}, & n > \text{sched_nr_latency} \end{cases}$$

The piecewise definition means: with at most 8 runnable tasks, the period is fixed at 6 ms and slices shrink as tasks are added. Past 8 tasks the slice is pinned at 0.75 ms and the *period stretches* linearly. This bounds context-switch overhead under load, at the cost of latency growing linearly with the runnable count — a deliberate throughput-versus-latency tradeoff.

These defaults are scaled by CPU count at boot. With `SCHED_TUNABLESCALING_LOG` (the default), `sched_init_granularity()` multiplies by $1 + \lfloor \log_2(\text{nrcpus}) \rfloor$, so a 64-CPU machine gets a 42 ms latency target rather than 6 ms. The reasoning is that bigger machines run more tasks and can absorb more per-task latency, and that switching cost scales with cache and NUMA depth.

6.2 Per-entity slice

The period is divided proportionally to weight:

$$\text{slice}_i = P(n) \cdot \frac{w_i}{\sum_j w_j}.$$

```
static u64 sched_slice(struct cfs_rq *cfs_rq, struct sched_entity *se)
{
    unsigned int nr_running = cfs_rq->nr_running;
    u64 slice;

    if (sched_feat(ALT_PERIOD))
        nr_running = rq_of(cfs_rq)->cfs.h_nr_running;

    slice = __sched_period(nr_running + !se->on_rq);

    for_each_sched_entity(se) {
        struct load_weight *load;
        struct load_weight lw;

        cfs_rq = cfs_rq_of(se);
        load = &cfs_rq->load;

        if (unlikely(!se->on_rq)) {
            lw = cfs_rq->load;
            update_load_add(&lw, se->load.weight);
            load = &lw;
        }
        slice = __calc_delta(slice, se->load.weight, load);
    }

    if (sched_feat(BASE_SLICE))
        slice = max(slice, (u64)sysctl_sched_min_granularity);

    return slice;
}
```

Note `for_each_sched_entity()`: with group scheduling the proportion is applied at *every level* of the cgroup hierarchy, so a task's slice is the product of its fractions all the way to the root. For a task at depth d ,

$$\text{slice} = P(n) \prod_{k=1}^d \frac{w^{(k)}}{W^{(k)}},$$

where $w^{(k)}$ is the entity's weight at level k and $W^{(k)}$ the total weight of its runqueue at that level.

The virtual equivalent, used when placing forked tasks:

```
static u64 sched_vslice(struct cfs_rq *cfs_rq, struct sched_entity *se)
{
    return calc_delta_fair(sched_slice(cfs_rq, se), se);
}
```

6.3 Accounting: update_curr()

This is the single most important function in the file. It is called from essentially every entry point — tick, enqueue, dequeue, task_fork, priority change, throttling — and it is where virtual time actually advances.

```
static void update_curr(struct cfs_rq *cfs_rq)
{
    struct sched_entity *curr = cfs_rq->curr;
    u64 now = rq_clock_task(rq_of(cfs_rq));
    u64 delta_exec;

    if (unlikely(!curr))
        return;

    delta_exec = now - curr->exec_start;
    if (unlikely((s64)delta_exec <= 0))
        return;

    curr->exec_start = now;

    schedstat_set(curr->statistics.exec_max,
                  max(delta_exec, curr->statistics.exec_max));

    curr->sum_exec_runtime += delta_exec;
    schedstat_add(cfs_rq->exec_clock, delta_exec);

    curr->vruntime += calc_delta_fair(delta_exec, curr);
    update_min_vruntime(cfs_rq);

    if (entity_is_task(curr)) {
        struct task_struct *curtask = task_of(curr);
        cgroup_account_cputime(curtask, delta_exec);
        account_group_exec_runtime(curtask, delta_exec);
    }

    account_cfs_rq_runtime(cfs_rq, delta_exec);
}
```

Three things worth noticing:

1. It uses `rq_clock_task()`, not `rq_clock()`. That is wall clock **minus** time stolen by IRQ and softirq handling and, on virtualized hosts, minus steal time. Tasks are not charged for interrupts they merely happened to be running during.
2. `curr->vruntime += calc_delta_fair(...)` is the equation of Section 3.3, and it is the only place ordinary execution advances vruntime.
3. `account_cfs_rq_runtime()` is the hook into bandwidth control (Section 10).

6.4 Tick-driven preemption

```
static void entity_tick(struct cfs_rq *cfs_rq, struct sched_entity *curr,
                      int queued)
{
    update_curr(cfs_rq);
    update_load_avg(cfs_rq, curr, UPDATE_TG);
    update_cfs_group(curr);
    /* ... */
}
```

```

    if (cfs_rq->nr_running > 1)
        check_preempt_tick(cfs_rq, curr);
}

static void check_preempt_tick(struct cfs_rq *cfs_rq, struct sched_entity *curr)
{
    unsigned long ideal_runtime, delta_exec;
    struct sched_entity *se;
    s64 delta;

    ideal_runtime = sched_slice(cfs_rq, curr);
    delta_exec = curr->sum_exec_runtime - curr->prev_sum_exec_runtime;

    if (delta_exec > ideal_runtime) {
        resched_curr(rq_of(cfs_rq));
        clear_buddies(cfs_rq, curr);
        return;
    }

    /*
     * Don't preempt below min_granularity -- churn isn't worth it.
     */
    if (delta_exec < sysctl_sched_min_granularity)
        return;

    se = __pick_first_entity(cfs_rq);
    delta = curr->vruntime - se->vruntime;

    if (delta < 0)
        return;

    if (delta > ideal_runtime)
        resched_curr(rq_of(cfs_rq));
}

```

Two independent preemption triggers, plus a floor:

- **Slice exhausted:** the current stint exceeded its computed share of the period.
- **Fairness violated:** the leftmost waiting entity is more than a full slice behind in vruntime. This catches the case where a high-weight task's slice is long but a newly-woken low-weight task is starving.
- **Granularity floor:** neither trigger fires below `min_granularity`, bounding switch rate.

`resched_curr()` sets `TIF_NEED_RESCHED`; the actual switch happens at the next preemption point (return to userspace, or a `preempt_enable()` on a preemptible kernel).

6.5 Wakeup preemption

A task waking up may preempt immediately — this is what makes interactive workloads responsive without any interactivity heuristic. `check_preempt_wakeup()` decides, using hysteresis:

```

static int wakeup_preempt_entity(struct sched_entity *curr,
                                struct sched_entity *se)
{
    s64 gran, vdiff = curr->vruntime - se->vruntime;

    if (vdiff <= 0)

```

```

        return -1;

    gran = wakeup_gran(se);
    if (vdiff > gran)
        return 1;

    return 0;
}

static unsigned long wakeup_gran(struct sched_entity *se)
{
    unsigned long gran = sysctl_sched_wakeup_granularity;
    return calc_delta_fair(gran, se);
}

```

That is, the wakee preempts only when

$$v_{\text{curr}} - v_{\text{wakee}} > \text{wakeup_gran} \cdot \frac{1024}{w_{\text{wakee}}}.$$

Without this buffer, two tasks ping-ponging on a futex or a pipe would context-switch on every wakeup and thrash the cache. The `calc_delta_fair()` scaling means a high-priority waker needs a smaller real-time lead to preempt.

`check_preempt_wakeup()` also handles the `WAKEUP_PREEMPTION` feature flag, `SCHED_IDLE` special-casing, and buddy hints.

6.6 Buddies

`cfs_rq->next`, `->last`, `->skip` are cache-locality hints that let `pick_next_entity()` deviate from strict leftmost order — but only slightly:

- **next**: set by `set_next_buddy()` on a wakeup. “This task was just woken by the current task; they are probably communicating, so run it next while the data is hot.”
- **last**: set in `check_preempt_wakeup()`. “This task was just preempted; prefer resuming it.”
- **skip**: set by `yield_task_fair()` to implement `sched_yield()`.

```

static struct sched_entity *pick_next_entity(struct cfs_rq *cfs_rq,
                                             struct sched_entity *curr)
{
    struct sched_entity *left = __pick_first_entity(cfs_rq);
    struct sched_entity *se;

    if (!left || (curr && entity_before(curr, left)))
        left = curr;
    se = left;

    if (cfs_rq->skip && cfs_rq->skip == se) { /* pick second-best */ }

    if (cfs_rq->last && wakeup_preempt_entity(cfs_rq->last, left) < 1)
        se = cfs_rq->last;

    if (cfs_rq->next && wakeup_preempt_entity(cfs_rq->next, left) < 1)
        se = cfs_rq->next;

    return se;
}

```

```
}
```

Crucially, a buddy is only honoured if `wakeup_preempt_entity(buddy, left) < 1` — that is, the buddy is not more than one granularity behind the true leftmost. Fairness deviation stays bounded. The heuristics are advisory; the invariant is not negotiable.

Toggle these at runtime via `/sys/kernel/debug/sched/features` (`NEXT_BUDDY`, `LAST_BUDDY`); the list is in `kernel/sched/features.h`.

7 Sleepers, forks, and migration

This is where classic CFS's main approximation lives, and where it is weakest.

7.1 The problem

A task that sleeps for 10 seconds does not advance its `vruntime`. Under strict GPS it would accrue 10 seconds' worth of credit and, on waking, would be entitled to monopolize the CPU until the debt cleared. That is catastrophic for the tasks already running.

Conversely, if you simply set a waking task's `vruntime` to `min_vruntime`, it gets zero credit for having slept, and interactive tasks — which sleep constantly — lose all responsiveness advantage.

CFS resolves this with a **bounded credit**.

```
static void place_entity(struct cfs_rq *cfs_rq, struct sched_entity *se,
                        int initial)
{
    u64 vruntime = cfs_rq->min_vruntime;

    /*
     * The 'current' period is already promised to the current tasks,
     * so charge a new task a full slice up front.
     */
    if (initial && sched_feat(START_DEBIT))
        vruntime += sched_vslice(cfs_rq, se);

    /* sleeps up to a single latency don't count. */
    if (!initial) {
        unsigned long thresh = sysctl_sched_latency;

        if (sched_feat(GENTLE_FAIR_SLEEPERS))
            thresh >>= 1;

        vruntime -= thresh;
    }

    /* ensure we never gain time by being placed backwards. */
    se->vruntime = max_vruntime(se->vruntime, vruntime);
}
```

7.2 Fork: START_DEBIT

A newly forked task is charged one full virtual slice before it runs:

$$v_{\text{new}} = \text{min_vruntime} + \text{vslice}(se).$$

Without this, `fork()` would be a way to reset your `vruntime` to `min_vruntime` and jump the queue — a fork bomb would also be a fairness attack. `task_fork_fair()` additionally does a `vruntime` swap with the parent under `sysctl_sched_child_runs_first`, so the child can be made to run first (useful for exec-heavy workloads, since the child usually calls `exec()` immediately and the parent’s page tables can be released sooner).

7.3 Wake: bounded sleeper credit

A waking task is placed according to

$$v_{\text{wake}} = \max(v_{\text{old}}, \text{min_vruntime} - \theta), \quad \theta = \begin{cases} \text{sched_latency}, & \text{plain} \\ \text{sched_latency}/2, & \text{GENTLE_FAIR_SLEEPERS} \end{cases}$$

`GENTLE_FAIR_SLEEPERS` is the default, and it is enabled because full credit caused latency spikes for the tasks that were *already running*.

The credit is **capped**: no matter how long you slept, you get at most θ worth of head start. And the outer max ensures a task’s `vruntime` never moves *backwards* — a task that slept only briefly keeps its own higher `vruntime` and gets no credit at all.

This gives the desired behaviour essentially for free:

- A text editor sleeping on keyboard input wakes with a small credit, becomes leftmost, pre-empts, handles the keystroke in microseconds, and sleeps again. It never uses its full slice, so it never falls behind — it stays perpetually near the front. Responsive.
- A compiler burns its full slice every time and steadily accumulates `vruntime`. It gets the CPU whenever the interactive tasks are asleep, which is most of the time. Throughput preserved.

No interactivity estimator. No sleep-average heuristic. The fairness invariant produces the behaviour the $O(1)$ scheduler needed 300 lines of tuned guesswork to approximate.

7.4 Migration normalization

`vruntime` is only meaningful relative to its own `cfs_rq->min_vruntime`, and different CPUs have wildly different values. So on `dequeue-for-sleep`, `dequeue_entity()` does

```
if (!(flags & DEQUEUE_SLEEP))
    se->vruntime -= cfs_rq->min_vruntime;
```

storing a *relative* `vruntime`, and `enqueue_entity()` re-bases it:

```
if (renorm && !curr)
    se->vruntime += cfs_rq->min_vruntime;
```

`migrate_task_rq_fair()` handles the cross-CPU case, subtracting the source `min_vruntime` and setting `ENQUEUE_MIGRATED` so the destination adds its own. Without this, migrating to an idle CPU with a low `min_vruntime` would be a free fairness windfall.

8 PELT: Per-Entity Load Tracking

`vruntime` answers “who should run next.” It says nothing useful about “how much CPU does this task actually need,” which is what load balancing and CPU frequency selection require. That is PELT’s job, added in 3.8 by Paul Turner and Ben Segall.

8.1 The geometric series

PELT divides time into periods of 1024 microseconds and maintains an exponentially-weighted moving average. Let u_i be the contribution in the i -th most recent period. The tracked signal is

$$L = \sum_{i=0}^{\infty} u_i y^i, \quad y = 0.5^{1/32} \approx 0.978572.$$

The choice $y^{32} = 1/2$ gives a **half-life of 32 ms**: a task's contribution from 32 ms ago counts half as much as now, from 64 ms ago a quarter, and so on. Fast enough to react to phase changes, slow enough not to be dominated by a single tick.

If a task runs continuously ($u_i = 1024$ for all i), the series converges to the maximum:

$$\text{LOAD_AVG_MAX} = 1024 \sum_{i=0}^{\infty} y^i = \frac{1024}{1-y} \approx 47742.$$

LOAD_AVG_MAX = 47742 is the constant you will see all over `pelt.c`. A fully-busy task has `util_avg` ≈ 1024 after normalization by `SCHED_CAPACITY_SCALE`.

8.2 Fixed-point implementation

There is no floating point in the kernel, so decay is a table lookup plus a shift. `decay_load()` computes $val \cdot y^n$ by splitting $n = 32q + r$ and using $y^{32q} = 2^{-q}$:

```
static u64 decay_load(u64 val, u64 n)
{
    unsigned int local_n;

    if (unlikely(n > LOAD_AVG_PERIOD * 63))
        return 0;

    local_n = n;

    /* y^32 = 1/2, so every 32 periods is one right shift */
    if (unlikely(local_n >= LOAD_AVG_PERIOD)) {
        val >>= local_n / LOAD_AVG_PERIOD;
        local_n %= LOAD_AVG_PERIOD;
    }

    val = mul_u64_u32_shr(val, runnable_avg_yN_inv[local_n], 32);
    return val;
}
```

`runnable_avg_yN_inv[]` is a 32-entry table of $y^n \cdot 2^{32}$, precomputed. So an arbitrary decay costs one shift, one table index, and one 64×32 multiply. The `n > LOAD_AVG_PERIOD * 63` early-out reflects that after 63 halvings the result is zero in fixed point anyway.

8.3 The three signals

`struct sched_avg` (in `include/linux/sched.h`) carries:

Field	Tracks	Used for
<code>load_avg</code>	<code>weight × time runnable</code>	Load balancing (weight-sensitive)
<code>runnable_avg</code>	time runnable (queued, on CPU or not)	Detecting CPU over-subscription
<code>util_avg</code>	time running (actually on CPU)	Frequency selection, capacity fitting
<code>util_est</code>	Estimated utilization (peak-holding)	Avoiding the ramp-up lag below

The distinction matters. `util_avg` saturates at CPU capacity — a task that wants twice a CPU still reports about 1024 — so it cannot express over-demand; that is what `runnable_avg` is for. And `load_avg` includes weight, so a nice-19 task hogging a CPU contributes little load even though it contributes full utilization.

`util_est` exists because of a real defect in the EWMA: a periodic task that sleeps long between bursts decays toward zero, so on each wake it looks tiny, `schedutil` picks a low frequency, and the task runs slowly until PELT ramps back up — for a signal with a 32 ms half-life, that ramp takes tens of milliseconds. `util_est` records the utilization observed at dequeue and holds it, so the task’s *known* demand is available immediately at wakeup. Added in 4.17 by Patrick Bellasi, and important for mobile and latency-sensitive workloads.

8.4 Entry points

`__update_load_avg_se()`, `__update_load_avg_cfs_rq()`, and `__update_load_sum()` in `pelt.c` do the work; `update_load_avg()` in `fair.c` is the wrapper called from `enqueue`, `dequeue`, `tick`, and the group-propagation paths. `propagate_entity_load_avg()` pushes changes up the cgroup hierarchy when a group’s composition changes.

The consumer on the frequency side is `cpufreq_update_util()`, leading to `schedutil_cpu_util()` in `kernel/sched/cpufreq_schedutil.c`, which maps aggregate utilization to a frequency request with a headroom factor (`map_util_perf()`, roughly $\times 1.25$). This is why scheduler and `cpufreq` stopped being independent subsystems.

9 Group scheduling

9.1 Motivation

Flat per-task fairness has an obvious hole: a user running 100 compile jobs beats a user running one editor, 100 to 1. Group scheduling (`CONFIG_FAIR_GROUP_SCHED`) makes fairness hierarchical — schedule *between groups* first, then within.

9.2 The recursion

Because CFS schedules `sched_entity`, not `task_struct`, the extension is structural rather than algorithmic. A struct `task_group` (in `sched.h`) holds, **per CPU**:

- `se[cpu]` — the entity representing this group on its parent’s runqueue
- `cfs_rq[cpu]` — the runqueue holding this group’s children on that CPU

`se->my_q` points down into the group’s own runqueue; `se->cfs_rq` points to the runqueue the entity is *queued on*. Picking becomes a descent:

```

do {
    se = pick_next_entity(cfs_rq, curr);
    set_next_entity(cfs_rq, se);
    cfs_rq = group_cfs_rq(se);    /* se->my_q */
} while (cfs_rq);

p = task_of(se);

```

The macro `for_each_sched_entity(se)`, which expands to `for (; se; se = se->parent)`, is the corresponding upward walk, used everywhere a change must propagate to the root.

9.3 The group weight problem, and its math

The hard part: what weight should `tg->se[cpu]` carry on CPU *cpu*?

The group has a global share `tg->shares` (set via `cpu.shares` or `cpu.weight` in `cgroup`). But the group's tasks are spread unevenly across CPUs. If a group with 4 runnable tasks has 3 on CPU 0 and 1 on CPU 1, its entity on CPU 0 must weigh more.

The ideal is to divide the global share in proportion to per-CPU load:

$$w_i^{ge} = \frac{tg \rightarrow shares \cdot grq_i \rightarrow load.weight}{\sum_{j \in CPUs} grq_j \rightarrow load.weight}.$$

The denominator is a sum over all CPUs, which cannot be computed on the enqueue fast path without a global lock. So the kernel approximates: each CPU maintains its contribution in `tg_load_avg_contrib` and atomically accumulates into `tg->load_avg` (via `update_tg_load_avg()`), giving

$$w^{ge} \approx \frac{tg \rightarrow shares \cdot grq \rightarrow load.weight}{tg \rightarrow load_avg}, \quad tg \rightarrow load_avg \approx \sum_j grq_j \rightarrow avg.load_avg.$$

`fair.c` carries an extended comment block deriving this and enumerating where the approximation breaks down — notably when a group's tasks are heavily concentrated on one CPU, and when `load_avg` lags a sudden change in group composition.

```

static long calc_group_shares(struct cfs_rq *cfs_rq)
{
    long tg_weight, tg_shares, load, shares;
    struct task_group *tg = cfs_rq->tg;

    tg_shares = READ_ONCE(tg->shares);
    load = max(scale_load_down(cfs_rq->load.weight), cfs_rq->avg.load_avg);
    tg_weight = atomic_long_read(&tg->load_avg);

    /* Ensure tg_weight >= load */
    tg_weight -= cfs_rq->tg_load_avg_contrib;
    tg_weight += load;

    shares = (tg_shares * load);
    if (tg_weight)
        shares /= tg_weight;

    return clamp_t(long, shares, MIN_SHARES, tg_shares);
}

```

}

`update_cfs_group()` calls this and feeds the result to `reweight_entity()`, which must carefully dequeue, adjust `cfs_rq->load`, and requeue — changing an entity’s weight while it sits in the tree would corrupt the parent’s load sum.

10 Bandwidth control

Fairness is *relative*. Sometimes you need an *absolute* cap: “this container gets 2 cores, even on an idle 64-core machine.” That is CFS bandwidth control (`CONFIG_CFS_BANDWIDTH`, Paul Turner, 3.2), the mechanism behind Kubernetes CPU limits.

10.1 Model

Each `task_group` gets a `cfs_bandwidth`:

- **period** (`cpu.cfs_period_us`, default 100 ms)
- **quota** (`cpu.cfs_quota_us`, `-1` meaning unlimited)
- **burst** (`cpu.cfs_burst_us`, added in 5.14) — allows carrying unused quota forward, up to a limit

The effective CPU cap is

$$C = \frac{\text{quota}}{\text{period}}.$$

Quota 200000 with period 100000 gives $C = 2$ CPUs.

10.2 Mechanism

A global `cfs_b->runtime` pool is refilled by an `hrtimer` each period (`sched_cfs_period_timer`). Per-CPU runqueues draw **slices** from it (`sched_cfs_bandwidth_slice_us`, default 5 ms) rather than taking a global lock per tick — that batching is essential for scalability.

`account_cfs_rq_runtime()`, called from `update_curr()`, decrements the local allocation. When it hits zero, `assign_cfs_rq_runtime()` tries to refill from the global pool; if the pool is exhausted, `throttle_cfs_rq()` removes the group’s entity from its parent runqueue entirely. Its tasks stay on `cfs_rq->tasks_timeline` but the group is invisible to the picker.

`unthrottle_cfs_rq()` reverses this at period boundaries. `return_cfs_rq_runtime()` and the slack timer (`sched_cfs_slack_timer`) return unused local slices to the global pool so idle CPUs do not strand quota.

10.3 The practical trap

The interaction of quota with multithreading is the single most common source of production confusion. With T runnable threads and quota Q over period P , the quota is exhausted after wall time

$$t_{\text{burn}} = \frac{Q}{T},$$

leaving the group throttled for $P - Q/T$. A container with $Q/P = 1$ CPU running an 8-thread JVM burns its 100 ms quota in 12.5 ms of wall time, then sits **throttled for 87.5 ms**. Average utilization is exactly 1 CPU as configured, but p99 latency is catastrophic.

`nr_throttled` and `throttled_time` in `cpu.stat` are the diagnostic. A 5.4-era fix by Dave Chiluk (returning unused per-CPU slack more aggressively) helped, but the structural issue remains. The usual advice is to size thread pools to the quota, or to use `cpu.shares / cpu.weight` instead of hard limits where possible.

11 SMP: load balancing and task placement

Everything above concerns a single runqueue. CFS keeps **one cfs_rq per CPU**, with no global lock and no global vruntime — fairness is per-CPU, and cross-CPU fairness is achieved statistically by balancing.

11.1 Scheduling domains

`struct sched_domain` (`include/linux/sched/topology.h`) forms a per-CPU hierarchy mirroring hardware topology: SMT siblings, then cores sharing L2, then LLC/package, then NUMA nodes, then across nodes. Each level has its own balancing interval and flags (`SD_SHARE_CPUCAPACITY`, `SD_SHARE_PKG_RESOURCES`, `SD_NUMA`, `SD_ASYM_CPUCAPACITY`). Migration gets progressively more expensive and progressively rarer as you go up. The hierarchy is built in `kernel/sched/topology.c`.

11.2 Wakeup placement

`select_task_rq_fair()` runs on every wakeup and is arguably more consequential for performance than periodic balancing:

1. **`wake_affine()`** — should the wakee run near the waker? If they share data (pipe, futex, producer/consumer), yes. `wake_affine_idle()` and `wake_affine_weight()` weigh cache benefit against load imbalance.
2. **`find_idlest_cpu()` / `find_idlest_group()`** — for `SD_BALANCE_FORK` and `SD_BALANCE_EXEC`, a full search of the domain. A fresh task has no cache footprint to preserve.
3. **`select_idle_sibling()`** — the fast path. Try the previous CPU, the waker's CPU, then scan the LLC domain via `select_idle_cpu()`. The scan is bounded by `SIS_PROP` heuristics tracking average idle time, because a linear scan of 128 CPUs on every wakeup is itself a scalability problem. `select_idle_core()` prefers a fully-idle core over a busy SMT sibling.

11.3 Periodic balancing

`run_rebalance_domains()` (softirq `SCHED_SOFTIRQ`) calls `rebalance_domains()`, which calls `load_balance()` per domain level.

`find_busiest_group()` and `update_sd_lb_stats()` classify each group into a `group_type`:

```
enum group_type {
    group_has_spare = 0,
    group_fully_busy,
    group_misfit_task,      /* task too big for this CPU's capacity */
    group_asym_packing,     /* ITMT / SMT packing preference */
    group_smt_balance,
    group_imbalanced,       /* affinity constraints block balancing */
    group_overloaded,
};
```

`calculate_imbalance()` then decides how much to move, and `detach_tasks()` / `attach_tasks()` do it. `can_migrate_task()` gates on affinity mask, cache hotness (`task_hot()`), using `sysctl_sched_migration_cost`, default 0.5 ms), and whether the task is currently running.

Since roughly 5.5 (Vincent Guittot’s rework), balancing is driven primarily by **utilization and runnable task counts** rather than raw `load_avg`. Weight-based load alone made the wrong call in common cases — one nice-19 spinner and one nice-0 spinner have very different loads but each fully occupies a CPU.

Related paths: `newidle_balance()` (a CPU about to idle pulls work), `nohz_idle_balance()` (an idle CPU balances on behalf of tickless ones), and `active_load_balance_cpu_stop()` (the migration thread forcibly moves a running task).

11.4 NUMA balancing

`task_numa_work()` periodically unmaps a task’s pages to force minor faults; `task_numa_fault()` records which node the task actually touches. Over time the scheduler builds a picture of each task’s memory affinity and migrates task and pages toward each other (`task_numa_migrate()`, `numa_migrate_preferred()`). Tasks sharing pages are grouped into a `numa_group` and placed together. Controlled by `/proc/sys/kernel/numa_balancing`.

12 Observability

```
# Global tunables (debugfs; formerly /proc/sys/kernel/sched_*)
ls /sys/kernel/debug/sched/
cat /sys/kernel/debug/sched/base_slice_ns      # 6.6+
cat /sys/kernel/debug/sched/latency_ns         # pre-6.6

# Feature flags -- toggle heuristics live
cat /sys/kernel/debug/sched/features
echo NO_GENTLE_FAIR_SLEEPERS > /sys/kernel/debug/sched/features

# Full runqueue dump: vruntime, load, PELT signals per task
cat /proc/sched_debug

# Per-task: sum_exec_runtime, run_delay (time waiting), nr_switches
cat /proc/<pid>/schedstat
cat /proc/<pid>/sched

# Tracepoints
perf sched record -- sleep 5 && perf sched latency
trace-cmd record -e sched:sched_switch -e sched:sched_wakeup \
                -e sched:sched_migrate_task
```

Useful tracepoints (include/trace/events/sched.h): `sched_switch`, `sched_wakeup`, `sched_migrate_task`, `sched_stat_runtime`, `sched_stat_wait`, `sched_stat_sleep`, `sched_process_exec`. The `pelt_*` tracepoints in `pelt.h` expose load-tracking internals.

Note that `schedstats` must be enabled (`CONFIG_SCHEDSTATS` plus `/proc/sys/kernel/sched_schedstats=1`) for wait-time accounting; it is off by default on many distributions because it costs a few percent.

13 EEVDF: what replaced CFS in 6.6

13.1 Why CFS was replaced

Three structural complaints accumulated over sixteen years:

1. **No latency interface.** A task could ask for *more CPU* (nice) but not for *sooner CPU*. Audio processing and a batch job might want equal throughput but wildly different wakeup la-

tency, and CFS had no vocabulary for that. Users resorted to abusing nice values, which changed the wrong thing.

2. **Unprincipled sleeper handling.** The bounded credit in `place_entity()` and the `GENTLE_FAIR_SLEEPERS` halving were tuned constants, not theory — the same category of heuristic CFS was created to eliminate.
3. **Heuristic accretion.** Buddies, wakeup granularity, `START_DEBIT`, `sched_nr_latency`: a growing pile of interacting knobs.

13.2 Lag and eligibility

EEVDF (Stoica and Abdel-Wahab, 1995) formalizes what CFS approximated. Define **lag** explicitly:

$$\text{lag}_i(t) = S_i^{\text{ideal}}(t) - S_i^{\text{actual}}(t).$$

Positive lag means under-served (owed CPU); negative means over-served. In virtual terms, with V the weighted-average virtual time,

$$\text{vlag}_i = V - v_i, \quad \text{lag}_i = w_i \cdot \text{vlag}_i.$$

A task is **eligible** if and only if $\text{lag}_i \geq 0$, that is $v_i \leq V$. This directly encodes “has not yet received its fair share.”

V is the weight-weighted mean of the runnable virtual times:

$$V = \frac{\sum_i w_i v_i}{\sum_i w_i}.$$

The kernel maintains this incrementally rather than recomputing. `cfs_rq->avg_vruntime` holds $\sum_i w_i(v_i - \text{min_vruntime})$ and `cfs_rq->avg_load` holds $\sum_i w_i$, both updated in `avg_vruntime_add()` and `avg_vruntime_sub()` on every enqueue and dequeue:

```
u64 avg_vruntime(struct cfs_rq *cfs_rq)
{
    struct sched_entity *curr = cfs_rq->curr;
    s64 avg = cfs_rq->avg_vruntime;
    long load = cfs_rq->avg_load;

    if (curr && curr->on_rq) {
        unsigned long weight = scale_load_down(curr->load.weight);
        avg += entity_key(cfs_rq, curr) * weight;
        load += weight;
    }

    if (load) {
        if (avg < 0)
            avg -= (load - 1);
        avg = div_s64(avg, load);
    }

    return cfs_rq->min_vruntime + avg;
}
```

}

`entity_eligible()` tests $v_i \leq V$ without doing the division, by cross-multiplying against `avg_load`.

13.3 Virtual deadlines

Each entity requests a **slice** r_i (`se->slice`). Its virtual deadline is

$$vd_i = ve_i + \frac{r_i}{w_i},$$

where ve_i is its virtual eligible time. The rule:

Among all *eligible* entities, run the one with the earliest virtual deadline.

This is what buys the latency interface. A task requesting a *smaller* slice gets a *nearer* deadline, so it is picked sooner — but it also runs for less time, so its long-run throughput share is unchanged.

Latency and bandwidth become independent axes. The slice is set via `sched_setattr()` with `sched_runtime`, or per-cgroup through the latency-nice interface.

13.4 The augmented tree

Finding “earliest deadline among eligible” is a two-dimensional query — the rbtree is keyed on `vruntime`, but we want to minimize deadline over a `vruntime`-bounded prefix. EEVDF augments each node with the minimum `vruntime` of its subtree (`se->min_vruntime`, maintained by `min_vruntime_update()` via `RB_DECLARE_CALLBACKS`), which lets `pick_eevdf()` prune subtrees containing no eligible entity and complete in $O(\log n)$.

13.5 Lag preservation

`place_entity()` was rewritten. Instead of the `thresh` heuristic, a sleeping task’s lag is **preserved** across the sleep (clamped to $\pm$ one slice) and restored on wakeup by setting

$$v_i = V - \text{vlag}_i.$$

`update_entity_lag()` computes and clamps it. A task that was over-served before sleeping wakes with negative lag and correctly waits its turn; a task that was under-served wakes eligible. The behaviour CFS approximated with tuned constants now falls out of the model.

13.6 What changed for users

CFS	EEVDF
<code>sysctl_sched_latency</code>	<code>sysctl_sched_base_slice</code> (<code>base_slice_ns</code> , about 0.70 ms)
<code>sysctl_sched_min_granularity</code>	removed
<code>sysctl_sched_wakeup_granularity</code>	removed
<code>GENTLE_FAIR_SLEEPERS</code> , <code>START_DEBIT</code>	removed
buddies (next / last / skip)	largely removed
latency only via nice abuse	<code>sched_attr.sched_runtime</code> , latency-nice

Nice values, weights, `calc_delta_fair()`, PELT, group scheduling, bandwidth control, and load balancing are all **unchanged**. If you learned CFS, you already know most of EEVDF.

14 Complexity summary

Operation	Cost	Where
Pick next entity (CFS)	$O(1)$	cached leftmost of rbtree
Pick next entity (EEVDF)	$O(\log n)$	<code>pick_eevdf()</code> , augmented tree
Enqueue / dequeue	$O(\log n)$	rbtree insert / erase
With group scheduling	$O(d \log n)$, $d = \text{cgroup depth}$	<code>for_each_sched_entity()</code>
<code>update_curr()</code>	$O(1)$	multiply-shift, no division
PELT decay	$O(1)$	table lookup plus shift
Load balance	$O(\text{CPUs})$ per domain level	amortized over the balance interval

The $O(1)$ scheduler was, as its name says, $O(1)$. CFS traded that for $O(\log n)$ and got a scheduler that is simpler, more predictable, and free of interactivity heuristics. For realistic runqueue lengths $\log n$ is 3 or 4 — a trade almost everyone agrees was correct.

15 Reading path

If you are going to read the source, this order works well:

1. Documentation/scheduler/sched-design-CFS.rst — Molnar’s design note, short
2. `include/linux/sched.h` — `struct sched_entity`, `struct sched_avg`
3. `kernel/sched/sched.h` — `struct cfs_rq`, `struct rq`, `struct task_group`
4. `kernel/sched/fair.c`, in this order:
 - `__calc_delta()`, `calc_delta_fair()` — the weight math
 - `update_curr()` — where virtual time advances
 - `update_min_vruntime()`, `entity_before()` — the ordering invariants
 - `enqueue_entity()`, `dequeue_entity()`, `place_entity()`
 - `pick_next_entity()` or `pick_eevdf()`
 - `check_preempt_tick()`, `check_preempt_wakeup()`
 - then the group-scheduling comment block above `calc_group_shares()`
5. `kernel/sched/pelt.c` — `decay_load()`, `__update_load_sum()`
6. `kernel/sched/core.c` — `__schedule()`, `pick_next_task()` for the class dispatch

The load-balancing half of `fair.c` (roughly `select_task_rq_fair()` onward) is a separate body of work and can be read independently.

15.1 Papers

- Parekh and Gallager, *A Generalized Processor Sharing Approach to Flow Control in Integrated Services Networks* (1993) — GPS and its fairness bounds
- Stoica and Abdel-Wahab, *Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation* (1995) — the EEVDF algorithm
- Demers, Keshav and Shenker, *Analysis and Simulation of a Fair Queueing Algorithm* (1989) — virtual time
- Lozi et al., *The Linux Scheduler: A Decade of Wasted Cores* (EuroSys 2016) — a well-known critique finding load-balancing bugs that left cores idle while work queued elsewhere; several fixes landed upstream

15.2 LWN articles worth finding

Search LWN for: “Completely Fair Scheduler”, “Per-entity load tracking”, “CFS bandwidth control”, “An EEVDF CPU scheduler for Linux”, “Scheduler wakeup path”, “Rethinking the scheduler load balancer”.